

Step-by-Step Implementation

STEP 1: Install Redis on Windows

Download Redis from Microsoft Archive

- <https://github.com/microsoftarchive/redis/releases>
- Extract to C:\Redis

Start Redis Server

```
cd C:\Redis  
redis-server.exe
```

Test Connection (in new terminal)

```
cd C:\Redis  
redis-cli ping  
Expected: PONG
```

STEP 2: Install Python Redis Package

Activate virtual environment

```
.\.venv\Scripts\activate
```

Install redis package

```
Uv addl redis
```

STEP 3: Create Redis Configuration

File: config/redis_config.py

```
# config/redis_config.py  
  
import redis  
from redis.exceptions import RedisError  
import os  
from dotenv import load_dotenv  
  
load_dotenv()
```

```
REDIS_HOST = os.getenv("REDIS_HOST", "localhost")
REDIS_PORT = int(os.getenv("REDIS_PORT", 6379))
REDIS_DB = int(os.getenv("REDIS_DB", 0))
REDIS_PASSWORD = os.getenv("REDIS_PASSWORD", None)

# Create Redis client
try:
    redis_client = redis.Redis(
        host=REDIS_HOST,
        port=REDIS_PORT,
        db=REDIS_DB,
        password=REDIS_PASSWORD,
        decode_responses=True, # Return strings instead of bytes
        socket_connect_timeout=5, # Timeout after 5s
        socket_timeout=5
    )

    # Test connection
    redis_client.ping()
    print("Redis connected successfully!")

except RedisError as e:
    print(f"Redis connection failed: {e}")
    print("Application will work without Redis (cache disabled)")
    redis_client = None

def get_redis_client():
    """Get Redis client instance"""
    return redis_client

def is_redis_available():
    """Check if Redis is available"""
    if redis_client is None:
        return False
    try:
        redis_client.ping()
```

```
        return True
    except RedisError:
        return False
```

STEP 4: Update .env File

Add Redis Configuration

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0
REDIS_PASSWORD=
```

STEP 5: Create Redis Service (Helper Functions)

File: utils/redis_service.py

```
# utils/cache.py

from config.redis_config import redis_client
import json

#get data from cache
def cache_get(key):

    if redis_client is None:
        return None

    value = redis_client.get(key)
    if value:
        try:
            return json.loads(value)    #it convert json string to python
object like dict,list
        except:
            return value
    return None

#set data to cache with expiration time (default 1 hour)
#setx means set with expiration time
```

#set means set without expiration time this is used for session management where we want to keep data until user logs out or session expires, but for 1000 user data it will consume more ram memory and can get crashed

```
def cache_set(key, value, expire_seconds=3600):

    if redis_client is None:
        return False

    serialized = json.dumps(value)      #it convert python object to json
string
    redis_client.setex(key, expire_seconds, serialized)
    return True

# Delete data from cache
def cache_delete(key):

    if redis_client is None:
        return False

    redis_client.delete(key)
    return True
```

STEP 6: Implement Caching in User Handler

File: handlers/user_handler.py

Example 1: Get User by Email (With Cache)

```
def get_user_by_email(db: Session, email: str):

    # Check cache first

    cached = cache_get(f"user_email:{email}")

    if cached:
```

```

        print(f"Cache hit: {email}")
        return User(**cached)

# Cache miss - query database
print(f"Cache miss: {email}")
user = db.query(User).filter(User.email == email).first()

# Cache the result (10 minutes)
if user:
    user_dict = {
        "id": user.id,
        "username": user.username,
        "email": user.email,
        "role_name": user.role_name
    }
    cache_set(f"user_email:{email}", user_dict, expire_seconds=600)

return user

```

Example 2: Show All Users (With Cache)

```

def show_all_users(db: Session, page: int, no_records: int, search: str,
current_user: User):

    # Create dynamic cache key
    cache_key =
f"users:{current_user.role_name.lower()}:{current_user.org_id or
'all'}:page:{page}:size:{no_records}"

    if search:
        cache_key += f":search:{search}"

    # Check cache first
    cached = cache_get(cache_key)

    if cached:
        logger.info(f"Cache hit: {cache_key}")

```

```

        return cached

    logger.info(f" Cache miss: {cache_key}")

    # Build query based on role
    if current_user.role_name.lower() == "super_admin":
        users_query = db.query(User)
    else:
        users_query = get_user_by_org_id(db, current_user.org_id)

    # Apply search filter
    if search:
        users_query = users_query.filter(
            (User.username.ilike(f"%{search}%")) |
            (User.email.ilike(f"%{search}%"))
        )

    # Get total count BEFORE pagination (Query object method)
    #count work with SQLAlchemy query object and does a fast COUNT(*) in
the database
    #count does not work with lists or already executed queries, it must
be called on the Query object before .all() or .first()
    total = users_query.count()

    if total == 0:
        result = {
            "page": page,
            "records": no_records,
            "total": 0,
            "pages": 0,
            "data": []
        }
        cache_set(cache_key, result, expire_seconds=300)
        return result

    # Calculate pages
    pages = (total + no_records - 1) // no_records

```

```

if page > pages:
    raise HTTPException(status_code=404, detail="Page out of range")

# Apply pagination and execute query (returns LIST of User objects)
start = (page - 1) * no_records
paginated_users = users_query.offset(start).limit(no_records).all()

# Convert SQLAlchemy User objects to dictionaries matching Userout
schema
# Userout has: id, username, email, org_id
users_data = []
for user in paginated_users:
    users_data.append({
        "id": user.id,
        "username": user.username,
        "email": user.email,
        "org_id": user.org_id
    })

# Build complete response matching Page[Userout] schema
result = {
    "page": page,
    "records": no_records,
    "total": total,
    "pages": pages,
    "data": users_data # List of dicts matching Userout
}

# Cache the complete result (10 minutes)
cache_set(cache_key, result, expire_seconds=600)
logger.info(f" Cached: {cache_key}")

return result

```